

QSys Raspberry Pi Setup

Use this guide when preparing a Raspberry Pi from a fresh checkout of the QSys source code.

What You Need

- Raspberry Pi OS or another Linux system with systemd
- Python 3.12 or newer
- `curl` or `wget` for installing `uv`
- Network access from the TV/browser to the Pi
- Up to three USB keyboard/keypad receivers
- The QSys source code copied or cloned onto the Pi

0. Setting up the Pi

0.1. Plug in the USB devices: Food, Drinks, and Chicken keypads, plus the dev keyboard if you are using one.

0.2. Connect to Wifi First, list all available Wi-Fi networks by typing:

```
nmcli device wifi list
```

Note the exact name (SSID) of your network, then connect using this command (replace `YOUR_SSID` and `YOUR_PASSWORD` with your actual network details):

```
nmcli device wifi connect "YOUR_SSID" password "YOUR_PASSWORD"
```

0.3 Update linux and install git if not already installed.

```
sudo apt update
sudo apt install -y git
git --version
```

1. Get The Source

```
git clone https://github.com/of89p/qsys.git
cd qsys
```

2. Install uv

Install `uv` with the official standalone installer from the `uv` installation docs:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

If `curl` is not installed, use `wget`:

```
wget -qO- https://astral.sh/uv/install.sh | sh
```

The installer usually places `uv` in `~/.local/bin`. Start a new shell, or make it available in the current shell:

```
export PATH="$HOME/.local/bin:$PATH"
uv --version
```

3. Install Python Dependencies

Use uv from the project directory:

```
uv sync
```

Or use standard Python tooling:

```
python3.12 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
```

The service installer prefers `.venv/bin/python` when it exists.

4. Plug In Keypads

Plug in the Food, Drinks, and Chicken keypads you plan to use. Check that Linux can see the input devices:

```
ls -l /dev/input/by-path/
```

The useful entries are the ones ending in `-event-kbd`. `/dev/input/by-path` is used because it identifies the USB port path, which is more useful than `/dev/input/by-id` when multiple keypads are the same brand. If you prefer the old `brand/device-id` paths, pass `--device-dir /dev/input/by-id` to `scripts/update_env_from_ls.py` or `scripts/install_systemd_services.py`. The setup scripts ignore `/dev/input/by-id/usb-Keychron_Keychron_K6-event-kbd` because that keyboard is reserved as the dev keyboard. When scanning `/dev/input/by-path`, the matching by-path symlink is ignored too.

5. Install Services

Run the installer from the project directory:

```
sudo python3 scripts/install_systemd_services.py
```

The installer:

- Detects this checkout path.
- Detects the Linux user that should run the services.
- Uses `.venv/bin/python` when available.
- Generates `.env` from `/dev/input/by-path` with Food, Drinks, and Chicken keypad paths.
- Renders and installs the systemd services.
- Adds the service user to the `input` group.
- Enables and restarts both services.

The installed services are:

```
qsys-server.service
qsys-interceptor.service
```

For a nonstandard install, pass explicit values:

```
sudo python3 scripts/install_systemd_services.py \
  --user pi \
  --root /home/pi/qsys \
  --python /home/pi/qsys/.venv/bin/python
```

Preview the generated service files without changing the system:

```
python3 scripts/install_systemd_services.py --dry-run
```

6. Set Up The Pi Display Browser

For the Pi display, autostart Chromium in kiosk mode with a dedicated local profile and the autoplay policy required for the notification sound:

```
mkdir -p ~/.config/qsys-chromium
mkdir -p ~/.config/labwc
nano ~/.config/labwc/autostart
```

Add this line:

```
chromium --kiosk --user-data-dir="$HOME/.config/qsys-chromium" --autoplay-policy=no-user-ges
```

If the command is named `chromium-browser` on your Pi, use that in the autostart line instead of `chromium`.

Each Pi should have its own local profile directory. You do not need to copy a profile between Pis; Chromium creates the profile automatically on first launch.

7. Check The Display

Find the Pi IP address:

```
hostname -I
```

Open this URL from the TV or another browser on the same network:

```
http://<pi-ip-address>:8080/
```

8. Test Without A Keypad

Submit a food queue number manually:

```
curl -X POST http://127.0.0.1:8080/api/queue \
  -H 'Content-Type: application/json' \
  -d '{"station": "food", "number": "12"}'
```

The display should show 012.

The valid `station` values are `drinks`, `chicken`, and `food`.

Keypad Assignment

The generated `.env` stores keypad paths:

```
FOOD_DEVICE_PATH=/dev/input/by-path/<food-keypad-event-kbd>
DRINKS_DEVICE_PATH=/dev/input/by-path/<drinks-keypad-event-kbd>
CHICKEN_DEVICE_PATH=/dev/input/by-path/<chicken-keypad-event-kbd>
```

By default, the first detected `*-event-kbd` device becomes `FOOD_DEVICE_PATH`, the second becomes `DRINKS_DEVICE_PATH`, and the third becomes `CHICKEN_DEVICE_PATH`. The generated paths normally begin with `/dev/input/by-path/`. Keep each keypad plugged into the same USB port so those assignments stay stable. The Keychron dev keyboard is excluded automatically and will not be assigned to Food, Drinks, or Chicken.

For a full three-keypad setup, regenerate `.env` with an explicit assignment order and restart the interceptor:

```
python3 scripts/update_env_from_ls.py --order food,drinks,chicken
sudo systemctl restart qsys-interceptor.service
```

Use the station order that matches the `ls -l /dev/input/by-path/` output. For older two-keypad Food/Drinks setups, `--swap` still swaps the first two detected devices. If fewer than three keypads are connected, the remaining device path variables stay empty.

Service Commands

Check service status:

```
systemctl status qsys-server.service
systemctl status qsys-interceptor.service
```

View logs:

```
journalctl -u qsys-server.service -f
journalctl -u qsys-interceptor.service -f
```

Restart after changing code or `.env`:

```
sudo systemctl restart qsys-server.service
sudo systemctl restart qsys-interceptor.service
```

Stop services:

```
sudo systemctl stop qsys-interceptor.service
sudo systemctl stop qsys-server.service
```

Updating A Pi

From the project directory:

```
git pull
uv sync
sudo python3 scripts/install_systemd_services.py
```

If you are not using uv, reinstall dependencies in the virtual environment:

```
. .venv/bin/activate
pip install -r requirements.txt
sudo python3 scripts/install_systemd_services.py
```

Troubleshooting

If the TV cannot load the page, confirm the server is running:

```
systemctl status qsys-server.service
```

If keypad input does nothing, check the interceptor logs:

```
journalctl -u qsys-interceptor.service -f
```

Then confirm `.env` points at current keyboard devices:

```
cat .env
ls -l /dev/input/by-path/
```

Regenerate `.env` after changing USB receivers:

```
python3 scripts/update_env_from_ls.py
sudo systemctl restart qsys-interceptor.service
```

If the interceptor reports permission errors, rerun the installer or add the service user to the `input` group:

```
sudo usermod -aG input "$USER"
sudo systemctl restart qsys-interceptor.service
```

If port 8080 is already in use, stop the other process or change the port in `app/server.py`.

Uninstall Services

Disable and stop both services:

```
sudo systemctl disable --now qsys-interceptor.service
sudo systemctl disable --now qsys-server.service
```

Remove the installed unit files:

```
sudo rm /etc/systemd/system/qsys-interceptor.service
sudo rm /etc/systemd/system/qsys-server.service
sudo systemctl daemon-reload
```